

ResQGrid AI — Complete Project Guide

Intelligent Emergency Resource Network | Product, user, developer, and operations documentation

Documentation edition 1.0 | Application version 0.1.0 | Source baseline: 0d85c00

Safety notice: ResQGrid AI is a demonstration and decision-support prototype for education, research, and hackathons. It is not certified or approved for real emergency response. Use fictional data only. AI outputs, scores, and travel estimates require human verification.

SOURCE-BASED EDITION · FICTIONAL DEMONSTRATION DATA ONLY

Contents

- 1. About this documentation 4
- 2. What is ResQGrid AI? 5
- 3. Problem, goals, and boundaries 6
- 4. Feature inventory 7
- 5. Users, roles, and permissions 9
- 6. End-to-end working process 10
- 7. Citizen and reporter guide 11
- 8. Dispatcher and administrator guide 12
- 9. Responder field workflow 13
- 10. Dashboard, map, and refresh behavior 14
- 11. AI triage and model behavior 15
- 12. Explainable priority scoring 16
- 13. Resource matching, hazards, and global optimization 17
- 14. Evidence, image analysis, and assistant 19
- 15. System architecture and data flow 20
- 16. Technology stack 21
- 17. Repository tour 22
- 18. Data model and state 23
- 19. API conventions and endpoint catalog 25
- 20. Safe API walkthrough 27
- 21. Security, privacy, and demo-access policy 28
- 22. Local prerequisites and configuration strategy 30
- 23. Windows local setup, step by step 31

24. Full Docker Compose workflow 33

25. Environment-variable reference 34

26. Hosted deployment and release checklist 36

27. PWA, offline outbox, and Android packaging 37

28. Demo accounts and sample dataset 38

29. Testing and verification 39

30. Operations, backup, and maintenance 40

31. Troubleshooting guide 41

32. Known limitations and honest presentation 43

33. Roadmap and extension priorities 44

34. Demo walkthrough and evaluation questions 45

35. Frequently asked questions 46

36. Glossary 47

37. Source references, license, and document maintenance 48

1. About this documentation

This guide explains the project from first principles through everyday use, internal algorithms, API integration, installation, deployment, security, testing, maintenance, and future development. It is based on the current application source, not solely on older pitch material or feature labels. The companion documentary script translates the same facts into a narrated demonstration.

The audience includes evaluators who need to understand the idea, operators learning the demonstration workflow, developers maintaining the code, and administrators preparing a controlled demo environment. Readers can follow chapters 2–14 for product understanding, 15–21 for implementation and integration, and 22–31 for setup and operations. Chapters 32–37 cover quality, limitations, roadmap, terminology, and traceability.

The Markdown file is the editable source for the PDF and Word editions. Regenerate exports after changing it; do not edit the exported files as the only source of truth. Documentation describes the source baseline above. Cloud deployments, model availability, pricing, and runtime data can change independently.

Status vocabulary used throughout: **Implemented** means connected to a current API or UI workflow; **API-only** means an endpoint exists but no dedicated dashboard control was found; **Configuration-dependent** means it needs external services or build configuration; **Planned** means a possible extension, not a delivered feature. Descriptive labels in old documentation do not override this distinction.

2. What is ResQGrid AI?

ResQGrid AI is a web-based emergency coordination demonstration. It collects incident reports, organizes them on a map and in a queue, assists a dispatcher with triage, calculates an explainable priority score, suggests resources, and records a human-approved assignment. Responders can then acknowledge and advance their assigned work through a field lifecycle.

In simple terms, it is a shared command dashboard that helps answer: What has been reported? Where is it? How serious might it be? Which resources are available? Why is a resource being suggested? Has a person approved the dispatch? What is the responder's latest recorded status?

Its operating principle is **AI recommends; humans approve**. Text and image analysis are advisory. The priority formula, resource matching, and global assignment optimizer are ordinary deterministic algorithms, not generative AI decisions. The application does not call emergency services, move vehicles, issue evacuation orders, or guarantee that a report reaches an official response organization.

A representative demonstration is a fictional flood report near Colombo. A citizen provides a location and description. A dispatcher reviews the report, runs triage, calculates priority, and requests resource suggestions. The dispatcher approves a suitable rescue resource, creating an assignment. A responder accepts the task, marks travel and arrival, and completes it. Completion returns the resource to the available pool; incident closure is a separate concern.

3. Problem, goals, and boundaries

Emergency coordination involves fragmented descriptions, uncertain locations, incomplete information, limited resources, and competing priorities. This project demonstrates how those inputs can be organized into one visible workflow. It is a software prototype of coordination concepts, not evidence that the approach improves real disaster outcomes.

The main goals are to standardize reports, expose useful context, make prioritization understandable, help compare available resources, retain human control, and provide traceable decision records. A responsive layout allows the same application to support desktop demonstrations and smaller field-device screens.

The present scope includes authenticated users, incident records, evidence images, resource and hazard registries, recommendations, assignments, selected audit events, and a command assistant. External weather feeds, sensor networks, hospital systems, telecom dispatch, continuous GPS telemetry, official incident verification, and road-network routing are not established integrations.

No performance improvement percentage, lives-saved claim, clinical accuracy, geographic coverage guarantee, or regulatory compliance claim is made. Example names and real geographic labels identify a fictional simulation, not actual incidents, official resource availability, or partnerships with the named organizations.

4. Feature inventory

Capability	Delivery status	What it currently does
Login and roles	Implemented	JWT login, profile lookup, and database-authoritative roles. Public registration creates citizens only.
Incident reporting	Implemented	Type, title, narrative, coordinates, people count, and medical-need input; additional structured fields through the API.
Command dashboard	Implemented	Incident queue, map, summary counters, detail workflow, mobile queue drawer, and role-based controls.
Map and location picker	Configuration-dependent	Google Maps when configured; MapLibre/OpenFreeMap fallback; pin selection, search, and browser geolocation.
Text triage	Implemented	Local English lexicon engine plus an optional configured LLM, combined through an ensemble.
Priority scoring	Implemented	Six weighted factors, numeric breakdown, reason codes, and persisted score.
Resource recommendations	Implemented	Availability/type/range filtering, approximate distance and hazard penalty, confidence ranking.
Human approval and dispatch	Implemented	Dashboard approval calls approval and assignment APIs sequentially.
Responder lifecycle	Implemented	Claim eligible work, accept, mark en route, mark on scene, and complete.
Image evidence	Implemented	Authorized image upload, sanitized storage, protected downloads, and preview.
Image interpretation	Configuration-dependent	Configured multimodal provider; no-key mode returns a clearly mock analysis.
Command assistant	Implemented	Dispatcher/admin questions answered using a bounded database context, optionally phrased by an LLM.
Global allocation optimizer	API-only	Hungarian assignment plan with a greedy comparison; does not dispatch resources.
Resource and hazard management	Primarily API	Create/list/update endpoints; map visibility is implemented, but full management screens are not present.
Audit browsing	API-only	Dispatcher/admin read access to recorded decision events; no dedicated audit dashboard was found.
Offline incident outbox	Implemented with limits	Queues report payloads in IndexedDB after connectivity failures; replays on application startup/reconnect.
PWA and Android wrapper	Configuration-dependent	Service-worker shell support and a Capacitor Android project using the same web UI.
WebSocket push, SMS, autonomous routing	Planned / not established	Do not present environment placeholders or UI wording as delivered integrations.

Installed packages and enum values do not by themselves prove functionality. For example, evidence enums include video/audio/document, but the upload implementation currently accepts only supported raster images. Hospital examples are stored as shelter-type resources, not in a dedicated hospital-capacity subsystem.

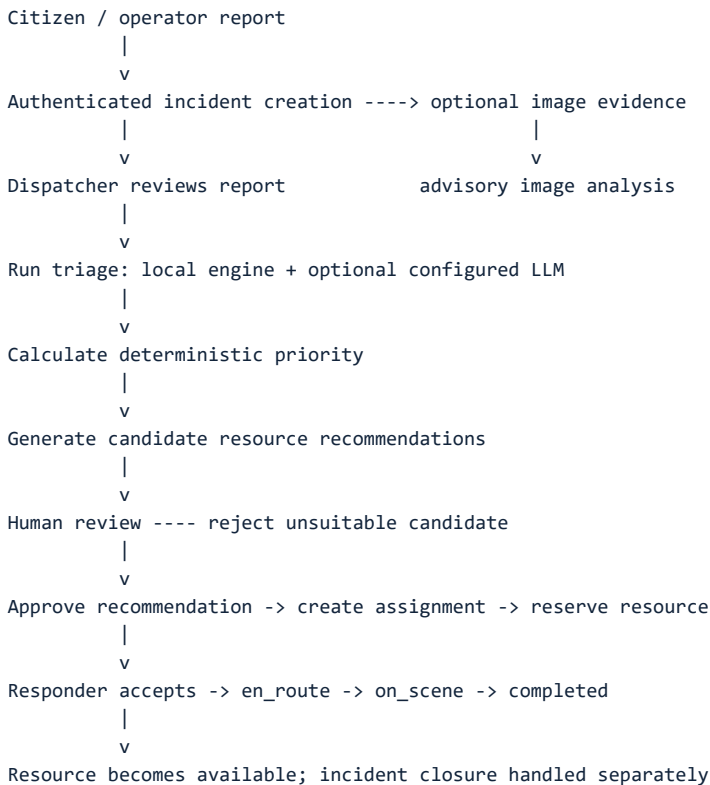
5. Users, roles, and permissions

Operation	Citizen	Responder	Dispatcher / admin
Create an incident	Yes	Yes	Yes
Read incidents	Own reports	Own reports, assigned work, and eligible unclaimed dispatches	All non-deleted incidents
Upload or analyze evidence	Own incidents	Own reports or assigned incidents; claim before writing to unclaimed work	All accessible incidents
Read evidence	Same incident read scope	Same incident read scope	All accessible incidents
Run triage, score, recommend	No	No	Yes
Approve/reject recommendations	No	No	Yes
Create assignment or optimize plan	No	No	Yes
Read assignments	No	Own and eligible unclaimed work	All
Advance assignment	No	Own work; may accept eligible unclaimed work	Yes, valid transitions only
Cancel assignment	No	No	Yes
Read resources and hazards	Yes	Yes	Yes
Create hazard report	Yes	Yes	Yes
Create/update resources; update hazards	No	No	Yes
Query command assistant; read audit logs	No	No	Yes
Read dashboard aggregate summary	Yes	Yes	Yes

The server enforces these rules independently of visible buttons. Incident-level denials commonly return 404 to avoid revealing inaccessible records. The database account role controls access; changing a role claim in a token does not grant privileges. Disabled or deleted accounts cannot authenticate successfully.

Dispatcher and admin share most implemented operational permissions. A complete administrator console, self-service privileged registration, password-reset flow, or role-management API is not part of the documented current routes. Dashboard aggregate counts are global, even when a citizen or responder can read only a subset of incidents; this is an important privacy and presentation limitation.

6. End-to-end working process



Each arrow represents application behavior or an explicit operator action, not an autonomous background agent. Creating a report does not automatically run triage or scoring. Uploading an image does not automatically rewrite the report's severity, priority, or resource assignment.

The dashboard's Approve button performs two HTTP requests: approve the recommendation, then create the assignment. These are not one database transaction. Approval may succeed while assignment fails because another operator already reserved the resource. Always inspect the assignment list after an error; an approved recommendation alone is not proof of dispatch.

The assignment endpoint permits an authorized dispatcher/admin to create a manual assignment without a recommendation ID. If an ID is supplied, the recommendation must be approved and match the incident/resource pair. Human authorization is required, but the backend does not enforce that every assignment originated from AI or a recommendation.

7. Citizen and reporter guide

Open the application, sign in, and select **Report New Incident**. Choose the incident type: flood, fire, landslide, earthquake, accident, medical, hazmat, infrastructure, or other. Enter a concise title and a factual description. Use wording such as “reported,” “visible,” and “unknown” to distinguish observation from assumptions.

Select the location by clicking the map, dragging the pin, searching for a place, or using the browser’s location permission. Review the latitude and longitude manually; the initial Colombo coordinates are a default, not an automatic determination of the incident location. Place search and map tiles require external connectivity. Browser geolocation normally requires HTTPS or a trusted local development context and user consent.

Enter people at risk only when known and mark medical need when appropriate. The API also supports vulnerable-person and injury counts, an address, and reporter contact fields; the current modal exposes a smaller subset. Submit once and wait for the result. If successful, the report appears in your accessible queue and can be selected for details.

To attach evidence, select the incident and use **Upload photo / screenshot**. Choose a non-sensitive JPEG, PNG, WebP, or GIF image. The system stores a sanitized PNG, so original metadata and animation should not be expected to survive. Uploads are online operations and are not included in the offline report outbox.

An offline message means the application attempted to queue the report on that device, not that a dispatcher received it. Keep the same account and browser context, reconnect, and verify the report appears server-side. Local storage failure, account switching, expired sessions, and browser storage cleanup can prevent delivery. Never rely on this prototype as an emergency reporting channel.

8. Dispatcher and administrator guide

Start with the incident queue and select a card or map marker. Review the original narrative, location, reported people, medical need, and evidence before requesting automated analysis. The visible queue orders loaded incidents by descending priority, then severity, then newest creation time. Severity filters do not search the entire database beyond the records already loaded.

Select **Run AI Triage**. Read the severity, model confidence, reason codes, and immediate needs. Treat these as advice and compare them with the report. Full ensemble diagnostics are stored in the incident's triage data, but the current panel does not provide a dedicated engine-disagreement visualization.

Select **Calculate Priority**. Inspect the numeric factor values and final score. Because time is an input, recomputing later can change the result even when the report text is unchanged. Scores are stored snapshots, not continuously recalculated urgency. Prefer the numeric values over visual bar lengths when explaining the calculation.

Select **Get Recommendations**. Review the resource name/type, confidence, approximate ETA, and compatibility reasons. Query the API for full constraints when needed; the current card shows only part of the recommendation data. Independently verify capacity, equipment, operating hours, access conditions, and suitability: these are not all hard constraints in the matcher.

Select **Approve** to approve and attempt dispatch, or **Reject** for an unsuitable candidate. The current Reject control sends the fixed reason "Not suitable"; richer explanations can be submitted through the API. Verify a dispatched-unit card exists and the resource is reserved. Do not repeatedly approve or generate recommendations to overcome a conflict without checking current state.

Use the command assistant for a situational summary, critical-incident questions, resource availability, hazard summaries, or pending approvals. It cannot replace direct record inspection. Resource/hazard administration, global optimization, audit browsing, and incident closure currently require API-level workflows where no dedicated UI is available.

9. Responder field workflow

Sign in as a responder and select an accessible incident with a dispatched unit. Newly dispatched work without a responder can be visible to eligible responders. The first successful **Mark Accept** operation claims that assignment under a database row lock. It is not a broadcast task assigned simultaneously to everyone.

Follow the sequence **assigned** → **accepted** → **en_route** → **on_scene** → **completed**. The interface presents the next valid action, such as **Mark En Route**, **Mark On Scene**, or **Mark Complete**. Record states only when the corresponding simulated step has actually occurred. API callers may include notes up to 5,000 characters.

The server rejects skipped/reversed transitions with 409. A responder cannot change another responder's assignment or cancel a dispatch. Dispatcher/admin cancellation is permitted from active states. Repeating the existing status returns the current record without applying a new note, so it is not a general notes-edit endpoint.

Acceptance, arrival, and completion timestamps are persisted. En-route does not have a dedicated timestamp field; the implementation uses the acceptance timestamp if one is missing. Completion or cancellation releases the resource only if its current assignment pointer still matches this assignment.

Assignment completion does not automatically resolve or close the incident, and assignment creation does not automatically advance incident status to assigned. These are separate stored state machines. No push, SMS, or radio notification integration was found; responders learn about work through application refresh and operator coordination, despite optimistic success-message wording.

10. Dashboard, map, and refresh behavior

The desktop view has a top KPI bar, left incident queue, central map, and right detail panel. On smaller screens, the queue becomes a slide-in drawer and selected details become an overlay. The map shows incident, resource, and hazard markers and supports incident selection. It is a spatial overview, not navigation guidance or continuous vehicle tracking.

The dashboard refreshes core data on mount, every 15 seconds, and after local actions. This is HTTP polling, not WebSocket streaming. The interval is not a delivery guarantee: API latency, a hidden tab, mobile suspension, network loss, or provider cold starts can add delay. Detail-panel recommendations, evidence, and assignments load on incident selection and relevant local actions; they are not all refreshed by the top-level interval. Reselect an incident if another user's changes appear stale.

The main incident request loads up to 200 records. The resource request uses the API default page size of 50. Counts on the map and queue therefore describe loaded data, while summary counters query the database globally. The assistant uses another bounded subset. These counts can legitimately differ and should not be presented as one consistent full-population view.

Map engine selection uses a configured Google Maps browser key when available and falls back to MapLibre/OpenFreeMap. Key restrictions, enabled APIs, billing, network access, and WebGL support can affect rendering. Map attributions must remain visible. External map and search services have their own terms and availability; do not promise unlimited or permanently free access.

MapLibre 6 requires the worker and its shared sibling module. The repository prepares both in `apps/web/public/maplibre/` through `predev/prebuild` hooks and sets the worker URL before either map initializes. Generated assets are ignored by Git and reproduced from the installed package. A blank map after a dependency/build change should trigger worker and tile diagnostics, not a broad weakening of the content security policy.

11. AI triage and model behavior

11.1 Local deterministic engine

Every triage starts with `local_engine.analyze_incident`. It examines English keyword/phrase lexicons, limited nearby negation, severity escalators/de-escalators, simple numeric count patterns, and structured report fields. A reported type receives a prior; stronger text evidence can propose a different type. Injuries can set medical need and elevate a low/medium local severity to high.

The local engine is server-side Python and needs no external AI key. “Offline local only” means no cloud model is used for that analysis; it does not mean the browser can run triage without reaching the API. Its confidence is a bounded heuristic based on detected signals, not a statistically calibrated reliability measure.

The English rules are intentionally simple. Substring overlap, limited negation, numbers attached to families/homes rather than people, and vulnerable-keyword frequency can produce misleading estimates. They do not provide validated Sinhala/Tamil understanding, medical diagnosis, or verified person counts. Structured positive counts can override extraction; zero-valued updates do not always override existing values in the current merge/persistence path.

11.2 Provider selection

The adapter factory chooses Gemini when a nonempty `GEMINI_API_KEY` is set. Otherwise it chooses Alibaba Model Studio when `DASHSCOPE_API_KEY` is nonempty and not the recognized placeholder. Otherwise it returns a mock adapter. There is no implemented automatic Gemini-to-Qwen retry chain.

Triage detects the mock adapter and uses the real local engine instead of the mock text completion. With a configured provider, it sends the report's title, narrative, type, coordinates, people/vulnerable/injury counts, and medical-need flag for JSON analysis. Provider exceptions and returned error dictionaries cause the service to proceed without an LLM result. Wrongly typed output can still fail merging or final validation; fallback is not a universal guarantee for every malformed response.

Defaults in source are `gemini-3.6-flash` and `qwen-plus`. These are configuration defaults, not a promise of provider availability. Confirm the model identifier, account region, permissions, quotas, and multimodal capability against the provider before use. Both text and image methods share the selected adapter's model setting.

11.3 Ensemble and stored output

With both engines, the ensemble chooses the higher severity, combines needs/reason codes, and records agreement. Severity distance of two or more levels sets a disagreement flag. Agreement weights are 50% severity, 30% type, and 20% medical-need agreement. The combined confidence starts at $1 - (1 - \text{local_confidence}) * (1 - \text{llm_confidence})$, can receive a small agreement bonus, and is reduced for strong severity disagreement.

Final triage data is validated using `TriageOutput`: severity must be low/medium/high/critical, confidence and evidence quality must be between 0 and 1, and people counts must be nonnegative or null. The route stores the validated result, confidence, reason codes, severity, selected structured values, immediate needs, evidence quality, timestamp, and triaged status. Ensemble details are stored under `incident.triage_data.ensemble`, not exposed as a separate field in the direct triage response.

A suggested incident type is retained in triage data; the route does not currently overwrite the incident's main `incident_type` field. Resource matching and environmental scoring use the main incident field. Operators must therefore not assume a displayed AI type refinement has changed all downstream calculations.

12. Explainable priority scoring

```
Priority = 0.30 * LifeRisk
          + 0.20 * MedicalUrgency
          + 0.15 * PeopleAtRisk
          + 0.15 * EnvironmentalRisk
          + 0.10 * TimeSensitivity
          + 0.10 * EvidenceConfidence
```

Factor	Current calculation	Default weight
Life risk	Severity maps to 25/50/75/100; positive injuries add 20, capped at 100.	0.30
Medical urgency	100 if incident medical need is true; otherwise 0; triage medical need raises it to at least 80.	0.20
People at risk	Count normalized against 50, plus vulnerable count normalized against 20, capped at 100.	0.15
Environmental risk	Fixed score by main incident type.	0.15
Time sensitivity	Age in minutes × severity multiplier, capped at 100; missing creation time yields 50.	0.10
Evidence confidence	Triage confidence × 100, with 0.5 fallback when the stored value is missing or zero.	0.10

Environmental scores are flood 80, fire 90, earthquake 95, landslide 75, hazmat 85, accident 50, medical 40, infrastructure 60, and other 50. Time multipliers are critical 3, high 2, medium 1, and low 0.5. The environmental factor is not a live weather feed or a spatial hazard calculation.

Worked example: assume a high-severity flood, no reported injuries, medical need true, 20 people at risk, four vulnerable people, age exactly 20 minutes, and triage confidence 0.8. The factors are life 75, medical 100, people 60, environment 80, time 40, and confidence 80. The weighted contributions are $22.5 + 20 + 9 + 12 + 4 + 8$, producing **75.5/100**. This example assumes those inputs at scoring time; a newly seeded or newly triaged incident need not have that score.

The engine returns score, components, weights, reason codes, and calculation time. The route persists the score/components and sets incident status to prioritized. Reasons include high life risk, urgent medical need, large population at risk, high environmental threat, time critical, high confidence evidence, and vulnerable population where thresholds are met.

The formula is explainable, not scientifically validated. A confidence of 80 is not 80% survival probability or 80% accuracy. “Evidence confidence” currently derives from text-triage confidence, not directly from uploaded-photo quality. Weights should be evaluated together and sum to 1 for the intended scale; current settings do not enforce that sum. Stored scores require deliberate recalculation as time or facts change.

13. Resource matching, hazards, and global optimization

13.1 Per-incident recommendations

The matcher selects non-deleted, available resources of preferred types. Flood prefers rescue boats, helicopters, and rescue teams; fire prefers fire trucks, rescue teams, and ambulances; medical prefers ambulances and medical teams. Other categories use the compatibility mapping in the service. If no preferred-type resource is available, the service falls back to any available type, which makes manual suitability review essential.

For each candidate it calculates Haversine distance from stored coordinates and skips a resource whose configured maximum range is exceeded. It estimates proximity of active hazards to the straight resource-to-incident segment using hazard radius, severity, and overlap. Summed hazard penalty is capped at 1. It is not a pathfinder and does not verify roads, bridges, travel modes, actual closures, or live traffic.

```
BaseMinutes = DistanceKm / 40 * 60
ETA = BaseMinutes * (1 + HazardPenalty) + HazardPenalty * 10
Confidence = BaseTypeConfidence
            - min(DistanceKm / 100, 0.3)
            - HazardPenalty * 0.35
BaseTypeConfidence = 0.9 for a preferred type, otherwise 0.5
Final confidence is floored at 0.1
```

At 10 km and penalty 0.2, ETA is $15 \times 1.2 + 2 = 20$ minutes. This is a heuristic estimate, not a promised arrival time. The same assumed speed is used across resource categories, so an ambulance and helicopter do not receive validated mode-specific travel estimates.

The service writes pending recommendation records, sorts candidates by confidence, returns up to five, and lists up to three alternative resource names. It persists candidates before truncating the returned list, so the dashboard's later list request may show more than five records. Repeated generation can add additional pending records rather than updating a single set.

Capacity, equipment, operating hours, and declared capabilities are useful context but are not comprehensively enforced as hard constraints. Hazard changes influence a later calculation; no automatic hazard-triggered rerouting or recommendation-refresh job was established. Review both original report and current registry state before dispatch.

13.2 Global assignment optimizer

POST /assignments/optimize is implemented for dispatcher/admin. It gathers incidents not resolved/closed, optionally filtered by IDs, available resources, and active hazards. It computes a Hungarian/Kuhn–Munkres minimum-cost matching and compares it with a row-order greedy baseline. No dedicated dashboard optimizer control is currently connected.

The pair cost is $\text{distance} * (1 + \text{hazard_penalty})$, plus a 25 km-equivalent penalty for a nonpreferred type. Exceeding maximum range is a hard violation. The pure-Python solver handles rectangular matrices, returns unmatched incidents, and does not mutate assignments or reserve resources. Its result remains an advisory plan requiring separate authorized dispatch actions.

The objective does not include the stored priority score, multi-resource needs, road travel time, validated capacity, or operational fairness. The incident query also does not explicitly exclude false alarms or already assigned incidents. A dispatcher must choose the eligible incident set carefully. "Savings" fields compare heuristic cost totals, not real kilometers driven, minutes saved, or verified response outcomes; comparisons

can be misleading if the plans serve different incident subsets.

14. Evidence, image analysis, and assistant

14.1 Evidence lifecycle

Evidence belongs to an incident and uploader. Before reading or writing it, the API checks incident access. Upload and quota checks are serialized by user/incident row locks. Defaults are 10 MiB per image, 20 million pixels, 100 MiB per uploader, and 50 evidence records per incident. Re-encoded output must also fit the size limit. Quota/count queries include stored rows regardless of soft-delete flag.

The storage layer decodes allowed raster formats and creates a fresh PNG, dropping metadata, comments, and appended bytes. This is a sanitized working copy, not an original forensic artifact with preserved EXIF or evidentiary chain of custody. A random storage key avoids using the submitted filename as the path. If database persistence fails after storage, the code attempts cleanup.

Storage uses OSS only when `APP_ENV` equals `production` and an OSS access key ID is configured; otherwise it uses the local upload directory, including in a production environment lacking OSS configuration. New OSS objects are private. Historical objects may need a separate access-control review. The browser downloads through `/evidence/{id}/content` with its bearer token and creates a temporary blob preview. There is no public `/uploads` directory mount.

14.2 Image interpretation

The server reads only authorized storage bytes and passes a data URL to the selected image adapter. Outputs include a scene description, detected signals, severity hint, estimated visible people, reasoning, confidence, and analysis time. They are stored on the evidence record; they do not automatically change incident triage, priority, or dispatch.

Without provider keys, image analysis returns the mock adapter's canned flood-related response. It is not actual local computer vision. A screenshot of that result must be labeled **mock analysis**. With a provider, results remain uncertain, and model/image support must be verified. Image JSON is normalized with default fields but does not have the same strict dedicated response schema as text triage.

An upload can succeed while automatic analysis fails; the route preserves the evidence and returns an analysis-unavailable indication. Inspect the stored image separately and retry analysis only when appropriate. Do not repeatedly upload the same image simply because analysis failed.

14.3 Command assistant

The assistant is restricted to dispatcher/admin. It gathers up to ten non-resolved/non-closed incidents ordered by priority, active hazards, available resources, and pending-approval counts. Without an AI key, it generates a database-derived response for recognized topics. With a configured model, it sends a summarized context; resource information supplied to that prompt is a count rather than the full inventory.

Answers may be cached for 120 seconds per process, keyed by normalized question. Provider failure triggers a 60-second process-wide cooldown and a database fallback. Source labels name registries, not necessarily individual clickable citations. Fixed confidence values in these responses are implementation labels, not calibrated truth probabilities.

Useful demonstration prompts are “Give me a situation summary,” “Which critical incidents need attention?”, “What resources are available?”, “What hazards are active?”, and “How many recommendations need approval?” The assistant cannot dispatch, change records, search arbitrary historical data, or promise that a summary represents every incident. Validate statements against records, especially after changes or when counts differ.

15. System architecture and data flow

```

Desktop browser / mobile browser / PWA / Capacitor WebView
|
+-- Next.js + React UI
|   +-- map engine -> external map tiles / Google Maps
|   +-- location search -> Nominatim
|   +-- IndexedDB -> queued incident payloads
|
+-- HTTPS JSON / multipart + bearer token
    |
    v
    FastAPI application (services/api/app)
    +-- auth, access policy, rate/body limits
    +-- incidents, evidence, resources, hazards
    +-- triage, priority, recommendations, optimizer
    +-- approvals, assignments, assistant, audit
    |
    +-- SQLAlchemy / asyncpg -> PostgreSQL
    +-- Redis -> shared request throttling
    +-- local evidence files OR private OSS
    +-- optional Gemini OR Model Studio

```

The working backend is a modular application under `services/api/app`, not a confirmed fleet of separately deployed AI/risk/allocation microservices. Routes delegate to services and pure algorithm modules. SQLAlchemy sessions normally commit after a successful request and roll back on exceptions; evidence upload has an explicit commit before optional analysis.

PostgreSQL stores relational records and JSONB outputs. The development database image includes PostGIS and initialization extensions, but core locations are float latitude/longitude columns and the current matching calculations run in Python. Do not infer spatial-index or PostGIS-routing usage from the installed extension alone.

Redis currently supports shared rate-limit counters, with a bounded process-local outage fallback. The assistant cache is in process memory, and the report outbox is in browser IndexedDB. A durable distributed task queue, separate background AI worker, vector database, retrieval embedding service, or event bus is not established in the current request path.

Trust boundaries are the browser/API interface, database, evidence store, external model providers, and map/search services. AI report content and image bytes can leave the API for a provider when configured. Map/search requests expose normal network metadata and requested locations/queries to those services. These flows matter even though the demonstration must use fictional data.

16. Technology stack

Layer	Verified source configuration	Responsibility
Web framework	Next.js dependency ^15.5.26; React ^19.3.0	App Router UI, production build, web/static export modes.
Frontend language/style	TypeScript; Tailwind CSS 3; Lucide icons	Typed client and command-dashboard styling.
HTTP client	Axios	Bearer attachment, API calls, 60-second request timeout.
Maps	MapLibre GL ^6.11.1; optional Google Maps	Basemap rendering, markers, location selection.
Mobile packaging	Capacitor ^8.5.2	Android WebView wrapper over exported web assets.
API framework	FastAPI 0.141.1; Uvicorn 0.34.0	Async HTTP routes and generated OpenAPI.
Validation	Pydantic 2.10.4; pydantic-settings 2.7.1	Schemas and environment settings.
Persistence	SQLAlchemy 2.0.36; asyncpg 0.30.0; Alembic 1.14.1	Async ORM and schema migrations.
Development infrastructure	PostgreSQL 16/PostGIS 3.4; Redis 7 images	Database/extensions and shared throttling.
Authentication	PyJWT 2.15.0; Passlib/bcrypt	HS256 tokens and password hashing.
AI integration	HTTPX REST adapters	Gemini and Model Studio-compatible APIs.
Evidence	Pillow 12.3.0; OSS SDK	Image sanitization and optional private object storage.
Quality gates	Pytest, Ruff, Mypy, Jest, TypeScript	Backend tests and scoped checks; web regression/build checks.
CI/runtime targets	Python 3.11; Node.js 24 in CI	Reproducible supported baseline.

Version ranges above are manifest declarations; the frontend lockfile determines exact transitive versions for `npm ci`. Do not upgrade all dependencies merely to follow this guide. Some installed libraries are unused or scaffolding-related; this table describes their established responsibilities rather than claiming every dependency is active.

17. Repository tour

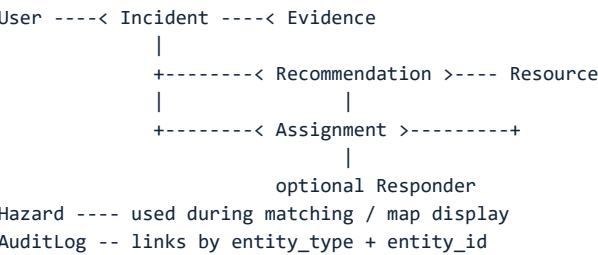
Location	What belongs here
<code>apps/web/src/app</code>	App entry, layout, and login page.
<code>apps/web/src/components</code>	Dashboard, queue, KPI bar, details, report modal, assistant, maps, and service-worker registration.
<code>apps/web/src/lib</code>	API client, auth context, map configuration/loader, browser outbox/sync, utilities, and regression tests.
<code>apps/web/public</code>	Manifest, service worker, icons, and generated MapLibre worker assets.
<code>apps/web/android</code>	Capacitor Android native project and Gradle configuration.
<code>apps/web/scripts</code>	MapLibre asset preparation and icon tooling.
<code>services/api/app/api/v1/routes</code>	HTTP endpoint handlers and per-route authorization.
<code>services/api/app/core</code>	Configuration, database sessions, tokens, dependencies, and rate limits.
<code>services/api/app/models</code>	Users, incidents, resources, recommendations, assignments, evidence, hazards, and audit records.
<code>services/api/app/schemas</code>	Request/response validation contracts.
<code>services/api/app/services</code>	Triage orchestration, priority, recommendations, assistant, vision, storage, and audit support.
<code>services/api/app/ai/triage</code>	Local text engine and ensemble merger.
<code>services/api/app/allocation</code>	Global assignment optimization.
<code>services/api/alembic</code>	Migration environment and schema revisions.
<code>services/api/tests</code>	Current executable backend test suite.
<code>db/init</code>	Development database extension initialization.
<code>.github/workflows/ci.yml</code>	CI gates for Python and web code.
<code>docs</code>	Setup references and this authoritative source-based guide/script.
<code>pitch</code>	Separate pre-existing pitch material; not replaced by this package.
<code>services/ai, services/risk, services/allocation, apps/mobile, infra, packages</code>	Scaffolding/extension locations; do not infer independent services or a separate mobile app.

Generated `.next`, `dist`, native build output, `node_modules`, virtual environments, and uploaded files are not application source. Private `.env` files, credentials, signing keys, and operational data do not belong in documentation exports or commits.

18. Data model and state

All core ORM entities inherit a UUID ID, creation/update timestamps, and soft-delete fields. Soft-delete support in a base model is not a general-purpose deletion API. PostgreSQL enums represent controlled states; JSONB stores structured analysis and flexible metadata.

Entity	Main data	Relationships
User	Email, username, password hash, role, profile, active flag	Reports incidents; may respond to assignments and approve recommendations.
Incident	Narrative, type, severity/status, coordinates, reporter, risk counts	Has evidence, recommendations, assignments, triage JSON, and priority data.
Resource	Type/status, coordinates, capacity/capabilities, organization, range	Referenced by recommendations and assignments; current assignment pointer.
Recommendation	Candidate resource, confidence, ETA, reasons, constraints, approval	Belongs to incident/resource; optional approver.
Assignment	Incident/resource/responder, optional recommendation, status/timestamps	Reserves one resource and records field progress.
Evidence	Storage reference, sanitized name/MIME/size, uploader, analysis JSON	Belongs to one incident.
Hazard	Type, title, center/radius, severity/status, reported-by	Used as contextual map data and by matching calculations.
Audit log	Action, entity reference, user, details, optional request/old/new fields	Polymorphic entity reference; not a strict foreign key to every target.



Incident statuses are reported, triaged, prioritized, assigned, in_progress, resolved, closed, and false_alarm. Recommendation statuses are pending, approved, rejected, and expired. The enum includes expired, but a scheduled expiration mechanism is not established. Assignment statuses follow chapter 9. Resource statuses are available, deployed, in_transit, maintenance, and offline. Hazard statuses are active, monitoring, and cleared.

Resource types include ambulance, fire_truck, rescue_boat, helicopter, rescue_team, medical_team, shelter, generator, supply_truck, drone, and other. Hazard types include flood, fire, landslide, road_blocked, chemical_spill, structural_collapse, power_outage, and other. Use these lowercase API values, not enum constant names.

Resource/hazard PATCH routes reuse their creation schemas: required identity/location fields are still required, and status is not included in those schemas. Do not document these as complete partial-update/status-management APIs. The resource's current_assignment_id is a stored UUID without a

declared foreign key in the ORM, so integrity also depends on application logic.

19. API conventions and endpoint catalog

The API prefix is `/api/v1`. Public metadata endpoints are `/`, `/health`, `/docs`, `/redoc`, and `/openapi.json`. The generated OpenAPI is the exact request/response contract for the deployed revision. Most requests use JSON; evidence uploads use multipart form data. Protected routes require `Authorization: Bearer <access_token>`.

In the tables, **Any** means authenticated; **D/A** means dispatcher/admin; **R/D/A** means responder/dispatcher/admin with object-level checks. Routes shown below are relative to `/api/v1`. List endpoints generally return arrays, not the generic paginated-envelope schema that also exists in the source.

Method	Path	Access	Purpose
POST	<code>/auth/register</code>	Public, throttled	Create citizen account.
POST	<code>/auth/login</code>	Public, throttled	Return access and refresh tokens.
POST	<code>/auth/refresh</code>	Refresh token, throttled	Issue tokens for an active account.
GET	<code>/auth/me</code>	Any	Current profile.
POST	<code>/incidents</code>	Any	Create report; optional client UUID for retries.
GET	<code>/incidents</code>	Any, scoped	Filtered/paginated incident array.
GET	<code>/incidents/{id}</code>	Any, scoped	Incident details.
PATCH	<code>/incidents/{id}</code>	D/A	Update supported report/status fields.
POST	<code>/incidents/{id}/triage</code>	D/A	Run and persist triage.
POST	<code>/incidents/{id}/priority</code>	D/A	Calculate and persist priority.
POST	<code>/incidents/{id}/recommendations</code>	D/A	Generate pending resource candidates.
POST	<code>/resources</code>	D/A	Create resource.
GET	<code>/resources</code>	Any	Filter by type/status; page and page_size.
GET	<code>/resources/available</code>	Any	Available resources, optional type.
GET	<code>/resources/{id}</code>	Any	Resource details.
PATCH	<code>/resources/{id}</code>	D/A	Update using ResourceCreate schema.
GET	<code>/recommendations</code>	D/A	Filter by status_filter/incident_id.
POST	<code>/recommendations/{id}/approve</code>	D/A	Approve pending recommendation.
POST	<code>/recommendations/{id}/reject</code>	D/A	Reject pending recommendation.
POST	<code>/assignments/optimize</code>	D/A	Advisory global matching plan.

Method	Path	Access	Purpose
POST	/assignments	D/A	Reserve available resource and create assignment.
GET	/assignments	R/D/A, scoped	Filter by incident_id/status_filter.
PATCH	/assignments/{id}	R/D/A, scoped	Claim/advance/cancel as permitted.
GET	/dashboard/summary	Any	Global summary counters; response-time field is null.
POST	/hazards	Any	Create hazard report.
GET	/hazards	Any	List non-deleted hazards with optional filters.
PATCH	/hazards/{id}	D/A	Update using HazardCreate schema.
GET	/audit/logs	D/A	Filtered/paginated audit events.
GET	/audit/logs/{id}	D/A	Audit event details.
POST	/assistant/query	D/A	Database-grounded assistant answer.
POST	/evidence	Any, write scope	Image upload; incident_id/analyze query parameters.
GET	/evidence/incident/{id}	Any, read scope	Incident evidence list.
GET	/evidence/{id}	Any, read scope	Evidence metadata.
GET	/evidence/{id}/content	Any, read scope	Authenticated file bytes; attachment/no-store.
POST	/evidence/{id}/analyze	Any, write scope	Re-run image analysis.

Incident listing supports status, severity, incident_type, page, and page_size. Incidents, resources, and audit logs use a default page size of 50 with a maximum of 200. Incident API order is newest-first; the UI separately sorts its loaded results by priority. Hazard listing is not automatically limited to active hazards unless filtered; matching explicitly queries active hazards.

Common responses: 200 successful read/update; 201 creation; 400 invalid business input; 401 invalid/missing authentication; 403 disallowed role/action; 404 missing or inaccessible record; 409 reservation/transition/ownership conflict; 413 body/image/quota limit; 415 unsupported image; 422 schema validation; 429 throttling; 502 unavailable image analysis; 500 unexpected server error. Use the actual response detail and request ID where available rather than assuming one error envelope everywhere.

20. Safe API walkthrough

Use only your isolated demonstration deployment. Obtain tokens through `/auth/login` using a demo or authorized test account, and keep them private. The following payloads are examples, not commands already executed. Swagger UI at `/docs` offers an Authorize control for the access token. Do not paste a refresh token as an access token.

Example incident creation:

```
{
  "title": "DEMO – Flooded access road",
  "description": "Fictional exercise: 20 people are stranded. Four vulnerable people need assistance.",
  "incident_type": "flood",
  "latitude": 6.9271,
  "longitude": 79.8612,
  "people_at_risk": 20,
  "vulnerable_people": 4,
  "medical_need": true
}
```

Record the returned incident ID. As dispatcher/admin, call its `/triage`, `/priority`, and `/recommendations` endpoints in order. Inspect the outputs. Approval uses `{"approved": true}`; rejection uses `{"approved": false, "reason": "Exercise: resource unsuitable"}`. An assignment payload is:

```
{
  "incident_id": "<returned incident UUID>",
  "resource_id": "<selected resource UUID>",
  "recommendation_id": "<approved recommendation UUID>"
}
```

Replace the angle-bracket placeholders with actual UUIDs before submission. Omitting `responder_id` leaves the assignment claimable according to the responder policy. As an eligible responder, patch `{"status": "accepted"}`, then `en_route`, `on_scene`, and `completed` in separate requests.

To request an optimizer plan, post `{"incident_ids": ["<incident UUID>"]}` to `/assignments/optimize`, or omit the filter to use the endpoint's default incident set. Review the plan without assuming it reserved anything. To close a completed exercise incident, an authorized dispatcher/admin can patch its status to `resolved` or `closed`; the current incident PATCH handler is not a rigorously enforced incident-transition state machine.

Idempotency applies to incident creation when the caller supplies an ID. The same ID owned by the same reporter returns the existing record with 200; it does not update the report with a changed payload. An ID owned by another reporter returns 409. Assignment creation does not have the same client-ID replay contract. After a timeout, inspect current assignments before retrying.

For PowerShell diagnostics, use an existing private token variable rather than hard-coding credentials into scripts:

```
$api = 'http://localhost:8000/api/v1'
$headers = @{ Authorization = "Bearer $accessToken" }
Invoke-RestMethod -Uri "$api/auth/me" -Headers $headers
Invoke-RestMethod -Uri "$api/incidents?page_size=50" -Headers $headers
```

21. Security, privacy, and demo-access policy

Passwords are hashed using bcrypt. Access and refresh JWTs have separate token types; required expiration/subject checks and database account validation protect route access. HS256 is the permitted algorithm. Production requires a non-placeholder random signing secret of at least 32 UTF-8 bytes. In development/test, a weak or absent secret is replaced with a process-generated secret, so restarts can invalidate tokens unless a stable local secret is configured.

Browser tokens are stored in localStorage, not HttpOnly cookies. This retains an XSS-related exposure. The refresh endpoint exists, but the current client does not automatically refresh on 401: it clears the session and returns to login. Logout removes local tokens; there is no established server-side refresh-token revocation list or single-use refresh rotation mechanism.

Request safeguards include role/object authorization, body-size limits, selected throttles, image decoding/re-encoding, storage-path validation, private evidence responses, browser/API security headers, explicit CORS origins, and generic unexpected-error messages. Header presence is defense in depth, not proof against every exploit. The browser CSP still permits inline scripts/styles for framework compatibility and allows specified map domains.

Operation	Default rate limit
Registration source	10 requests per hour.
Login/refresh source	60 requests per five minutes.
Login account identity	20 attempts per five minutes.
Incident creation per user	30 requests per minute.
Evidence upload per user	30 requests per hour.
Evidence analysis per user	30 requests per hour.

Redis provides shared counters. If unavailable, limits use a bounded per-process fallback, which is not globally consistent across multiple instances. Source identity depends on trusted reverse-proxy configuration; do not blindly trust arbitrary forwarded IP headers. General JSON mutation bodies are bounded at 1 MiB; the upload route permits configured file size plus 64 KiB multipart overhead.

The public demo accounts and login buttons are intentionally retained. Anyone knowing the demo admin/dispatcher credentials can exercise broad demo privileges. This is not an appropriate access policy for real data, even after technical hardening. Keep the environment clearly labeled, isolated, and limited to synthetic material. A future restricted deployment requires a separately authorized account/seeding policy change.

Audit events exist for incident creation, triage, prioritization, recommendation generation, and recommendation approval/rejection. No audit-edit/delete route is exposed. This is not comprehensive auditing of every mutation, tamper-evident storage, or cryptographic immutability. Assignment changes and several registry/PATCH operations do not call the same audit service in current handlers.

Do not upload personal medical information, identifiable victim photographs, private location histories, tokens, or provider credentials. Provider prompts can contain report data; logs in some AI paths can include questions or exception text. Establish retention, redaction, consent, provider agreements, and deletion processes before any future sensitive-data use. This guide does not certify GDPR, HIPAA, public-safety, or

emergency-management compliance.

22. Local prerequisites and configuration strategy

For a reproducible local demo, use Git, Docker Desktop with Compose, Python 3.11, Node.js 24, and the committed npm lockfile. Android packaging additionally requires Android Studio, its compatible JDK/SDK, and SDK platform 36. Development database ports default to 5432 and 6379; API/web ports default to 8000 and 3000.

The simplest controllable development arrangement is PostgreSQL/Redis in Docker, with API and web processes running locally. Full Compose is also available. This guide gives Windows PowerShell commands; commands in the legacy Makefile generally assume a POSIX shell. Do not use `&&` with older Windows PowerShell.

Three configuration contexts must be distinguished: the repository-root `.env` is used by Docker Compose substitution; `services/api/.env` is read when the API runs with that working directory; `apps/web/.env.local` configures Next.js. Shell environment variables can override file values. Frontend `NEXT_PUBLIC_*` values are public build inputs, not secret storage.

Do not copy the entire root example into the API directory without filtering it: it contains Compose/frontend/SMTP fields not declared in the backend Settings model. Use a minimal backend-only file. Existing private configuration must be preserved; edit deliberately rather than overwriting it with sample values. No private environment values were used to write this guide.

23. Windows local setup, step by step

23.1 Get the project and start infrastructure

For a new authorized checkout only:

```
git clone https://github.com/deshanlakshitha/resqgrid-ai.git
Set-Location .\resqgrid-ai
```

For the supplied workspace, open PowerShell in the project root. Docker Desktop must be running. Review the root example and configure local database credentials consistently if changing the defaults. Start only infrastructure:

```
docker compose up -d postgres redis
docker compose ps
```

Wait until both services are healthy. Default database/Redis published ports are bound to loopback. If you changed the Compose database user, password, database, or port, use the same values in the API connection URL. These development defaults must not become publicly exposed production credentials.

23.2 Prepare the API

In a dedicated terminal:

```
Set-Location '.\services\api'
py -3.11 -m venv .venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

Create or edit `services/api/.env` with backend settings only. A minimal local example using unchanged Compose defaults is:

```
APP_ENV=development
APP_DEBUG=false
DATABASE_URL=postgresql+asyncpg://resqgrid:resqgrid_dev_password@localhost:5432/resqgrid_ai
REDIS_URL=redis://localhost:6379/0
CORS_ORIGINS=http://localhost:3000
GEMINI_API_KEY=
DASHSCOPE_API_KEY=
LOG_LEVEL=INFO
LOG_FORMAT=json
```

For a stable local signing secret without printing it, set a process environment value before starting the API. Store a persistent secret through your own secure configuration process if required; a new value invalidates earlier tokens.

```
$env:JWT_SECRET = (& .\venv\Scripts\python.exe -c "import secrets;
    print(secrets.token_urlsafe(48))")
.\venv\Scripts\python.exe -m alembic upgrade head
.\venv\Scripts\python.exe -m app.seed
.\venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Check each command's exit status before continuing; older PowerShell does not automatically stop on every native executable failure. Migrations and seeding must target your local database, never a production database by accident. Virtual-environment executables are used directly, so changing PowerShell's execution policy is unnecessary.

23.3 Prepare the web application

In another terminal, open the repository root, then:

```
Set-Location '.\apps\web'  
npm ci
```

Create or edit `apps/web/.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1  
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=  
NEXT_PUBLIC_MAP_STYLE_URL=https://tiles.openfreemap.org/styles/liberty
```

Start with `npm run dev`. The predev hook prepares MapLibre worker assets. Open <http://localhost:3000>, sign in using a demo account, and verify the map, queue, and a report workflow. If using `127.0.0.1` instead of `localhost` in the browser or a different port, add that exact origin to API CORS configuration and restart the API.

The API metadata is at <http://localhost:8000/health>; interactive documentation is at <http://localhost:8000/docs>. A healthy metadata response is not a full database readiness test. Confirm login and an authorized list request too.

24. Full Docker Compose workflow

From the root, configure the Compose `.env` for your isolated demo. The API service receives explicit environment entries; not every Settings field is forwarded by the existing Compose file. Start infrastructure first, initialize the database with a one-off API container, and then start the application services:

```
docker compose up -d postgres redis
docker compose build api web
docker compose run --rm api alembic upgrade head
docker compose run --rm api python -m app.seed
docker compose up -d api web
docker compose ps
```

The development Compose API command overrides the image startup script with reload-mode Uvicorn, so do not assume it automatically migrates/seeds. Within containers the database/Redis hostnames are `postgres` and `redis`; the browser-facing API URL normally remains `http://localhost:8000/api/v1` when browsing on the host computer.

Use `docker compose logs --tail=100 api` and the corresponding web command for diagnostics, taking care not to publish sensitive logs. `docker compose down` stops and removes the service containers/network while preserving named volumes. Do not add volume-removal options unless intentionally discarding the demo database. The Makefile's `reset/clean` targets are destructive and are not troubleshooting defaults.

Compose is a development topology with bind mounts and reload commands. It is not a hardened multi-instance production recipe. Changing upload limits, OSS settings, token lifetimes, or other unforwarded backend settings in the root `.env` alone may have no effect; explicitly configure the API container environment for those options in a controlled deployment.

25. Environment-variable reference

25.1 API settings

Setting	Default / meaning	Handling
APP_ENV	development; production enables strict secret validation and conditional OSS selection	Set deliberately.
APP_DEBUG	false in Settings	SQL logging follows this value; keep false outside local troubleshooting.
APP_HOST / APP_PORT	0.0.0.0 / 8000	Process commands may supply host/port separately.
APP_SECRET_KEY	Legacy placeholder setting	Not the JWT signing secret; do not confuse it with JWT_SECRET.
DATABASE_URL	Async PostgreSQL URL	Secret; PostgreSQL URLs are normalized to asyncpg form.
REDIS_URL	redis://localhost:6379/0	Secret if credentials included; shared rate limiting.
JWT_SECRET	No fixed safe default	Required strong random secret for production.
JWT_ALGORITHM	HS256	Other algorithms rejected.
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	30	Access lifetime.
JWT_REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh lifetime.
GEMINI_API_KEY / GEMINI_MODEL	Empty key / gemini-3.6-flash	Secret key; selected before Model Studio.
DASHSCOPE_API_KEY / MODEL_STUDIO_MODEL	Empty key / qwen-plus	Secret key; verify model capabilities.
MODEL_STUDIO_BASE_URL	DashScope compatible-mode v1 endpoint	Match provider region/account.
OSS_ACCESS_KEY_ID / OSS_ACCESS_KEY_SECRET	Empty	Server-only secrets; private bucket access.
OSS_BUCKET / OSS_REGION / OSS_ENDPOINT	resqgrid-evidence / ap-southeast-1 / regional OSS endpoint	Endpoint/bucket determine actual storage access.
UPLOAD_DIR	services/api/uploads resolved from code	Needs durable writable storage when local mode is used.
MAX_UPLOAD_BYTES	10,485,760	Configurable from 1 KiB to 25 MiB; UI still checks 10 MiB.

Setting	Default / meaning	Handling
MAX_IMAGE_PIXELS	20,000,000	Upper configurable bound 40,000,000.
EVIDENCE_USER_QUOTA_BYTES	104,857,600	Per-uploader stored-byte quota.
EVIDENCE_INCIDENT_LIMIT	50	Configurable from 1 to 500.
CORS_ORIGINS	http://localhost:3000	Comma-separated exact origins; wildcard rejected.
LOG_LEVEL / LOG_FORMAT	INFO / json	Structured logging; sanitize before sharing.

Priority weights are `WEIGHT_LIFE_RISK=0.30`, `WEIGHT_MEDICAL_URGENCY=0.20`, `WEIGHT_PEOPLE_AT_RISK=0.15`, `WEIGHT_ENVIRONMENTAL_RISK=0.15`, `WEIGHT_TIME_SENSITIVITY=0.10`, and `WEIGHT_EVIDENCE_CONFIDENCE=0.10`. Change them only as a versioned experiment with documented rationale and tests, not as validated emergency policy.

25.2 Web, packaging, and template settings

`NEXT_PUBLIC_API_URL` must include `/api/v1` and an origin reachable from the end user's device.

`NEXT_PUBLIC_GOOGLE_MAPS_API_KEY` is necessarily visible to browsers; protect it with provider-side origin/API restrictions. `NEXT_PUBLIC_MAP_STYLE_URL` selects the command-map fallback style; the location picker imports the fixed fallback constant, so do not assume this variable changes every map. `BUILD_FOR_CAPACITOR=true` selects static export; otherwise the web uses standalone output.

`PORT` is consumed by the backend startup script on hosting platforms. Root `POSTGRES_*` and `REDIS_*` fields support Compose substitution, not an alternative to the backend's connection URLs. `NEXT_PUBLIC_WS_URL` and `SMTP` values appear in older templates but do not establish a working WebSocket or email subsystem. Do not put provider/DB/OSS secrets in any `NEXT_PUBLIC_*` variable.

26. Hosted deployment and release checklist

The previously verified demonstration endpoints are the Vercel web app at

<https://web-two-mu-edisuy5td1.vercel.app/> and Render API at <https://resqgrid-api-9wns.onrender.com/api/v1>.

The repository is <https://github.com/deshanlakshitha/resqgrid-ai>. These are reference locations, not an uptime or certification guarantee. Documentation generation does not publish a new release.

For Vercel, import the repository with root directory `apps/web`, use the existing Next.js build configuration, match Node.js 24, and set public API/map configuration before building. The prebuild worker hook must run. Changing public variables requires a new build; a backend-only restart cannot change an already compiled client bundle.

For Render, the blueprint uses the API Dockerfile from the repository root, Singapore region, main branch, and `/health`. Set database, Redis, JWT, CORS, and optional provider/storage settings in the hosting environment. The Docker startup script runs Alembic, invokes demo seeding, and then Uvicorn on the platform PORT. Migration failure stops startup. Production Settings rejects weak signing secrets.

The checked-in `render.yaml` CORS value names an older frontend deployment, so verify and supply the actual current origin in the service environment. A generic database URL may need provider-specific TLS/network configuration. Persistent PostgreSQL, Redis access, and durable evidence storage must be provisioned separately; the blueprint does not prove those services or backups are configured.

A free/ephemeral container filesystem is not durable evidence storage. With no configured OSS key the API can still save uploads locally, even in production, and lose them on rebuild/restart. Configure supported private OSS or a durable mounted upload directory before relying on retained demo images. The existing blueprint does not automatically configure OSS.

Release checklist: review intended changes and secrets; run tests/type checks/build; verify migrations and recovery plan; configure exact CORS/API origins; preserve the intended demo-access policy; deploy only after authorization; verify health plus authenticated reads; check MapLibre worker/shared files and map tiles; exercise a fictional report/approval/assignment; check protected evidence denies unauthenticated access; record the released commit. Use hosting rollback for code when appropriate, but review schema compatibility before reverting database changes.

Do not assume a GitHub CI pass blocks every hosting deployment unless that gate is configured. Monitor CI and hosting status separately. No new commit, push, or deployment is required merely to read or generate these documents.

27. PWA, offline outbox, and Android packaging

27.1 Browser installation and offline limits

The web app includes a manifest, icons, and service worker. Browser installation availability depends on browser/platform support and HTTPS. The worker caches shell/navigation and eligible static assets; it deliberately excludes API calls, authenticated requests, cross-origin traffic, and upload paths. It does not create an offline copy of the incident database or evidence collection.

Offline report submission is a separate IndexedDB outbox. The client generates a UUID, tries the network, and queues on failures without an HTTP response. Server validation/auth/server-error responses are surfaced instead of queued. Replay runs on application startup and online events, using the same UUID. It is not a guaranteed operating-system background task while the app is closed.

Known limits: the outbox is not partitioned by user, IndexedDB failures can be silently ignored, server errors can leave an item waiting for another replay trigger, and replay currently treats any 409 as already recorded even though the API also uses 409 for cross-user ownership conflict. Avoid shared-browser account switching with queued reports. Verify successful server delivery before clearing site data. Local-only triage still needs the server; offline maps, evidence upload, dispatch, and assistant responses are not guaranteed.

27.2 Android build

Capacitor uses app ID `ai.resqgrid.app`, app name `ResQGrid AI`, and web directory `dist`. The checked-in Android project has minimum SDK 24 and compile/target SDK 36. The source under `apps/mobile` is not a separate implemented native application.

From `apps/web`, with dependencies installed and Android Studio/JDK/SDK configured, build static web assets against a reachable API, sync them, and assemble a debug package:

```
$env:BUILD_FOR_CAPACITOR = 'true'
$env:NEXT_PUBLIC_API_URL = 'https://resqgrid-api-9wns.onrender.com/api/v1'
$env:NEXT_PUBLIC_GOOGLE_MAPS_API_KEY = ''
npm run build
npx cap sync android
.\android\gradlew.bat -p .\android assembleDebug
```

Stop if any command fails. The debug APK is expected at

`apps/web/android/app/build/outputs/apk/debug/app-debug.apk`. The provided `build-android.ps1` automates this sequence with `ApiUrl/Release` parameters. `prepare-android-studio.ps1` emulator uses 10.0.2.2 to reach the development host; device mode detects a LAN address. A physical phone's localhost is the phone itself, not your PC.

Prefer HTTPS. A local HTTP API may require explicitly reviewed Android cleartext-network settings and firewall access; do not globally weaken network security to make a demo work. Add the actual WebView origin to the API's CORS allowlist where needed. Release builds require proper signing, protected signing material, testing, and distribution approval; an unsigned release APK is not an app-store release.

Use a fresh terminal for normal web work, or clear build-only environment overrides, then rebuild in web mode. Static export and web builds have historically shared intermediate output; build the intended final target last. A passing web export is not proof that a native APK was assembled or tested on a device.

28. Demo accounts and sample dataset

The following intentionally public credentials are already present in the login page/seed source. They are included for the existing demonstration workflow only; none is a production-safe password.

Role	Email	Demo password
Dispatcher	dispatcher@resqgrid.local	dispatch123
Administrator	admin@resqgrid.local	admin123
Responder 1	responder1@resqgrid.local	respond123
Responder 2	responder2@resqgrid.local	respond123
Citizen	citizen@resqgrid.local	citizen123

The seed defines five users, twenty incidents, thirty resources, and eight hazards centered around Colombo. Five shelter examples and three hospital examples are included within the thirty resources, not additional separate tables. Hospital examples use the shelter resource type. Locations and scenario names are illustrative, not authoritative dispatch coordinates.

The seeder checks whether the demo admin email exists and skips the whole seed if it does. It is not a complete repair/upsert mechanism for a partially populated database. Seed incidents are reported-state records with predefined severities and counts; they do not all begin with calculated triage or priority. Current hosted counts can differ because users may have exercised the application.

Use an isolated local dataset for recording and repeatable tests. Prefix new reports with DEMO, use non-sensitive images, and keep track of generated incident/assignment IDs. Do not reset a shared database to reproduce a screenshot. Startup seeding and public demo access remain intentionally unchanged by this documentation package.

29. Testing and verification

During preparation of this documentation, the isolated backend suite passed **100 tests**, and the frontend regression suite passed **13 tests**. The frontend run emitted a non-blocking Jest naming-collision warning because a generated standalone package and the project package share a name. Tests still completed successfully; a clean CI checkout does not contain that generated standalone tree before the test step.

Test area	Existing coverage focus
test_security.py	Registration, JWT/role/object policy, assignments, evidence, limits, storage and related regressions.
test_triage_ensemble.py	Local analysis and ensemble behavior.
test_priority_engine.py	Deterministic factor calculations and priorities.
test_optimizer.py	Hungarian/greedy matching and plan behavior.
test_ai_adapter.py	Adapter behavior/factory cases.
test_assistant_fallback.py	Database-derived assistant responses and failure handling.
test_schemas.py	Request/response validation examples.
apps/web/src/lib/security.test.js	Service-worker exclusions, MapLibre worker assets, and dependency-tool compatibility.

Run from services/api: `.\.venv\Scripts\python.exe -m pytest tests -q`. Run from apps/web: `npm test -- --runInBand`, `npm run type-check`, and `npm run build`. The backend test configuration isolates settings and avoids loading developer secrets. Existing tests are not a substitute for real database integration, concurrency, device, or end-to-end load tests.

GitHub CI uses Python 3.11 and Node.js 24. Backend lint/type checks target selected AI/allocation/security-test modules, not every Python file. CI runs the full backend test directory with coverage reporting. Web CI runs npm ci, regression tests, TypeScript checks, and a build. A job title is not proof that a repository-wide lint or a specific coverage target is enforced.

Before this documentation task, source baseline 0d85c00 had successful web and Capacitor static-export builds and CI. The map repair was verified using real components with synthetic fixture data and protected worker/shared assets. That earlier check did not provide a native-browser screenshot or a fresh native Android APK test. These historical results are not claimed as new deployment tests performed for documentation.

Recommended manual acceptance: sign in with each role; create one fictional report; verify scoped visibility; run triage/priority/recommendations; approve one candidate; claim with one responder; confirm a second responder cannot modify it; complete and verify release; inspect evidence access; test one reconnect replay; compare dashboard counts with pagination scope; verify all maps render streets and markers. Run shared-state/concurrency checks only against an authorized disposable environment.

30. Operations, backup, and maintenance

Monitor API availability, authenticated list success, database connection/pool errors, Redis fallback warnings, AI provider failures/rate limits, evidence-storage errors, quota usage, and frontend map/worker fetch failures. `/health` returns a static healthy payload and does not actively check PostgreSQL, Redis, OSS, or AI dependencies. Startup logs a database failure but does not make the health response a readiness guarantee.

Use structured request IDs to correlate application logs where supported. Avoid publishing raw prompts, incident text, account data, or provider exceptions. A real log-redaction and retention policy is an operational requirement, not a feature guaranteed by the current logger. The project does not include a verified monitoring dashboard, on-call policy, or automated alerting configuration.

Back up PostgreSQL and evidence storage together with a manifest of the deployed version and relevant non-secret configuration. Prefer provider-native database snapshots and private object-versioning/lifecycle controls where available. Encrypt backups, restrict access, and perform a restore rehearsal into a separate environment. A database-only backup cannot recover lost local images. Browser outboxes are device-local and not part of the server backup.

Schedule migrations as controlled release steps. The existing startup script migrates every time it starts; before adding multiple instances, design migration coordination and avoid concurrent schema changes. Do not downgrade a populated database as a routine rollback. Verify compatibility, restore strategy, and data-loss impact first.

Dependency maintenance should use supported tool versions and the committed lockfile. Review overrides and run compatibility tests after updates. The MapLibre worker/shared pair must match the installed main library. Rotate real secrets through deployment configuration and re-authenticate users after JWT secret changes. Keep demo policies explicit and separate from any future restricted-data deployment.

No measured capacity, uptime SLA, recovery-time objective, or recovery-point objective is established. Set and test those values before claiming operational suitability. Hosting and AI costs depend on plan, region, traffic, storage, and provider quotas; no monthly budget or free-tier guarantee can be inferred from this source.

31. Troubleshooting guide

Symptom	Likely check	Safe response
Login fails immediately	API URL, database, seed status, credentials	Verify health plus database-backed request; use intended demo account; inspect sanitized errors.
Session expires after restart	Development-generated JWT secret	Configure a stable private local secret and sign in again.
401 after token expiry	Client has no automatic refresh	Sign in again; API integrations may use the refresh endpoint correctly.
CORS error	Exact scheme/host/port mismatch	Update the API allowlist, especially stale Render blueprint origin; restart API.
Frontend still calls old API	Public variable embedded at build	Rebuild frontend with correct NEXT_PUBLIC_API_URL.
Blank/faint map without streets	Worker/shared module, tiles, WebGL, CSP	Verify both /maplibre assets return JS/200; rebuild with hooks; hard refresh; inspect browser network errors.
Google map rejected	API enablement, origin restrictions, billing	Correct key configuration or use the configured fallback; do not expose server secrets.
Location search unavailable	Search-provider/network limits	Enter reviewed coordinates manually; never assume the default pin is correct.
AI text provider fails	Key/model/region/quota/response shape	Inspect diagnostics; local triage or assistant fallback may remain available; label mode accurately.
Photo always describes a flood	No provider configured	That is mock vision output, not actual image interpretation.
Upload returns 413/415	Size, pixel limit, quota, valid image	Use a smaller supported raster image; check server settings and stored quota.
Image missing after redeploy	Ephemeral local storage	Restore authorized backup; configure durable storage for subsequent uploads.
Approval exists but no assignment	Second request failed/conflicted	Inspect resource and assignments; resolve explicitly through API, not repeated approval clicks.
Resource or incident seems stale	Polling/detail reload/pagination	Reselect incident or refresh; compare API state and list scope.
Completed unit, incident still active	Separate status machines	Authorized operator resolves/closes incident separately when appropriate.
Offline report not delivered	IndexedDB/session/replay/account mismatch	Keep same account, reconnect/reopen, verify server record; preserve queued data during diagnosis.
429 response	Throttle reached or fallback bounded	Respect Retry-After; avoid retry loops.
npm ci lock mismatch	Different npm versions/optional entries	Match Node 24/current compatible npm, inspect lockfile; regenerate deliberately and retest.

Symptom	Likely check	Safe response
Jest duplicate package warning	Generated .next/standalone/package.json	Use a clean checkout/workspace for CI-style tests; do not edit dependency code.
Native device cannot reach API	Phone localhost/LAN/firewall/HTTPS	Use reachable HTTPS API or reviewed emulator/LAN setup; verify CORS and network policy.
Environment validation fails	Extra keys or weak production secret	Use backend-only settings and a strong private JWT secret.
Health passes while data calls fail	Static health response	Diagnose database/auth/storage directly; health alone is insufficient.

For the MapLibre regression, the important URLs are `/maplibre/maplibre-gl-worker.mjs` and `/maplibre/maplibre-gl-shared.mjs`. The worker imports its sibling, so publishing only one file is insufficient. Browser refresh cannot repair a deployment that never generated the assets; build first, then refresh. Do not disable CSP or authentication to troubleshoot rendering.

32. Known limitations and honest presentation

This is a demonstration prototype with intentionally public privileged demo access. It is not a certified emergency system and must not handle real personal/medical emergency data. The project's proprietary license also means a public repository is not permission to reuse or redistribute it without authorization.

Operational gaps include polling rather than push, limited/paginated client datasets, static health metadata, incomplete state synchronization between incident and assignment, sequential approval/dispatch requests, no documented notification channel, and no validated high-availability/backups/load capacity.

Resource/hazard management and audit/optimizer features are partly API-only.

Algorithmic gaps include heuristic English triage, uncalibrated confidence, malformed-response edge cases, nonvalidated priority weights, time-dependent stored scores, approximate straight-line ETA/hazard penalties, incomplete capacity/capability enforcement, and optimizer objectives without incident priority. None should be described as clinical judgment, safest routing, guaranteed accuracy, or measured operational savings.

Offline support is partial: shell caching and attempted report queuing do not guarantee cold offline startup, account-safe replay, background delivery, offline basemaps, or evidence retention. Image uploads are sanitized derivatives; vision can be mock and is not incident truth. Assistant answers use a bounded context and can be temporarily stale.

Security improvements do not constitute a guarantee of vulnerability freedom. LocalStorage tokens, lack of refresh revocation, selective audit coverage, historic object ACLs, possible log exposure, and operational secret/storage policies remain relevant considerations. No new application fix is implied by identifying these documentation boundaries.

When presenting, say "suggested resource" instead of "proven optimal rescue," "estimated travel time" instead of "live guaranteed ETA," "database-grounded summary" instead of "knows every incident," and "selected audit records" instead of "tamper-proof record of every action." Show implemented behavior and clearly label mock/future material.

33. Roadmap and extension priorities

These are proposed work items, not committed delivery dates or features already shipped. Prioritize correctness and testability before expanding visual polish or adding more providers.

First, improve workflow integrity: make approval/dispatch recoverable or atomic, align incident/assignment state transitions, provide explicit resolution/cancellation controls, prevent duplicate pending recommendations, and expose structured errors/retry guidance. Add end-to-end tests with real PostgreSQL row-lock/concurrency behavior.

Second, strengthen report reliability: partition outboxes by user, surface storage failures, distinguish ownership conflicts from replay success, add user-visible delivery status, and design supported background sync. Test mobile suspension, account changes, private browsing, storage pressure, and expired credentials.

Third, improve decision support: validate triage against a labeled evaluation set, test negation/count handling and local languages, separate confidence from severity and evidence quality, add configurable hard resource constraints, and integrate a real road-routing service with verified hazard data. Evaluate fairness, capacity, multiple resources per incident, and priority-aware optimization before claiming better allocation.

Fourth, complete operator tooling: add management screens, audit viewer, optimizer plan review, ensemble-disagreement display, pagination/search, stale-data indicators, provider-mode labels, and accessible keyboard/screen-reader behavior. Implement notifications only with explicit delivery/acknowledgment semantics.

Finally, address deployment maturity: controlled account provisioning, token lifecycle improvements, comprehensive audit/redaction policies, durable storage/backups, readiness checks, alerting, scaling, recovery drills, signed mobile releases, privacy/legal review, and independent safety/security evaluation. Any transition beyond a synthetic-data demonstration requires a separate requirements and authorization process.

34. Demo walkthrough and evaluation questions

A reliable demonstration can use one fictional flood scenario and three roles. Prepare a fresh or known isolated dataset, a harmless sample image, stable connectivity, and an available rescue resource. Rehearse before recording. Keep API credentials and developer tools containing secrets out of the captured screen.

Run sequence: show the login safety context; report the fictional flood as citizen; open it as dispatcher; demonstrate map/queue selection; run triage; explain numeric priority; show image analysis with its actual mode; generate suggestions; explain approximate ETA; approve once; switch to responder; accept/travel/arrive/complete; return to dispatcher; verify resource release; show assistant summary; finish with limitations. Use separate browser profiles/devices for concurrent roles because tabs share localStorage.

For an API-only optimizer demonstration, use Swagger or a sanitized saved response clearly labeled with its source/mode. Do not animate a nonexistent dashboard button. For no-key mode, label text as local triage and image output as mock. If live provider results differ from rehearsal, explain the uncertainty rather than editing the narration to claim a fixed result.

Useful evaluation questions: Can users distinguish a report from verified fact? Are dispatch reasons understandable? Does role isolation work? Can the second responder improperly claim work? Do operators notice stale information? Can queued reports be accounted for? Are image/model limits clear? Can a deployment be restored? These are evaluation goals; answering them requires evidence, not only a successful slide presentation.

The companion documentary script includes a timed narrative, shot directions, recording checklist, and short cut-down. It is a production script, not a rendered video or a claim that footage has already been captured.

35. Frequently asked questions

Is this a real emergency service? No. It is an educational/research/hackathon decision-support prototype. For a real emergency, use official local emergency channels.

Does AI automatically dispatch a team? No. The current dashboard requires an authorized human approval action. The server also permits authorized manual assignment without a recommendation ID.

Can the project run without a paid AI account? Text triage and database-derived assistant responses can run without a cloud AI key. Image analysis is then mock. Database/API access and external map connectivity are separate requirements.

Is Qwen always the active model? No. Gemini is chosen first when its key is configured; otherwise Model Studio, then mock adapter. Service-specific fallback behavior differs.

Is a higher confidence result always more severe? No. Severity, priority, recommendation confidence, and model confidence are different quantities. None is a certified outcome probability.

Is the map a routing engine? No. It displays coordinates and contextual markers. Matching uses approximate distance and hazard overlap, not street navigation.

Does “completed” close the incident? No. It completes the assignment and releases its resource; the incident remains separately managed.

Why do counts differ? API scope, frontend pagination, global KPI counts, assistant context limits, polling, and caching are different mechanisms.

Can I upload a PDF or video? Not through the current evidence upload path. It accepts validated JPEG/PNG/WebP/GIF images and stores a sanitized PNG.

Can I reuse the repository commercially? The LICENSE states all rights reserved and requires prior written permission. Public accessibility is not an open-source license.

36. Glossary

Term	Meaning in this project
Incident	A reported event needing assessment; not necessarily independently verified.
Triage	Structured advisory interpretation of a report's type, severity, needs, and confidence.
Priority	Stored numeric result of the six-factor heuristic.
Resource	A vehicle, team, equipment item, or shelter-type registry record.
Recommendation	A pending/approved/rejected candidate resource proposal.
Assignment	An authorized resource reservation with a responder lifecycle.
Hazard	A reported contextual threat/obstacle represented by location/radius and severity.
Human-in-the-loop	An operator reviews and authorizes consequential actions.
Ensemble	Combination of local and optional LLM triage outputs.
LLM	External language model accessed through an adapter when configured.
Hungarian algorithm	Deterministic minimum-cost bipartite matching algorithm.
Haversine distance	Great-circle distance between coordinates; not road distance.
ETA	Approximate travel-duration heuristic in this application.
JWT / RBAC	Signed token authentication / role-based access control.
Object authorization	Checking access to the particular incident/evidence/assignment, not just the route.
Idempotent replay	Reusing a report UUID to avoid a second record for the same owner.
PWA / Capacitor	Browser-installable web capability / native wrapper for web assets.
JSONB / Alembic	PostgreSQL structured JSON storage / database migration tooling.
CORS / CSP	Browser cross-origin access policy / content security policy.
OSS	Alibaba Cloud Object Storage Service, optionally used for private evidence.

37. Source references, license, and document maintenance

Current implementation is the authority for behavior; generated OpenAPI is the exact API contract for a deployed revision. Older README, setup, security, architecture, and pitch files remain useful historical references but can contain outdated versions or aspirational features. Where they differ from this guide, recheck the source before presenting the claim.

Topic	Primary repository source
Routes and application lifecycle	services/api/app/main.py; services/api/app/api/v1/router.py; services/api/app/api/v1/routes/
Roles and object scope	services/api/app/core/deps.py; services/api/app/core/security.py; auth/assignment/evidence routes
Settings and limits	services/api/app/core/config.py; services/api/app/core/rate_limit.py; services/api/app/middleware/security.py
Triage and providers	services/api/app/services/triage_service.py; services/api/app/ai/triage/; services/api/app/adapters/ai_adapter.py
Scoring and matching	services/api/app/services/priority_service.py; services/api/app/services/recommendation_service.py
Global optimizer	services/api/app/allocation/optimizer.py; services/api/app/api/v1/routes/assignments.py
Evidence and assistant	services/api/app/services/storage_service.py; vision_service.py; assistant_service.py in the same directory
Data contracts	services/api/app/models/; services/api/app/schemas/schemas.py; services/api/alembic/versions/
User workflow	apps/web/src/components/Dashboard.tsx; DetailPanel.tsx; ReportIncidentModal.tsx; IncidentQueue.tsx
Maps	apps/web/src/lib/mapConfig.ts; apps/web/src/components/map/; LocationPickerMap.tsx; apps/web/scripts/prepare-maplibre.mjs
Auth and offline support	apps/web/src/lib/api.ts; auth.tsx; outbox.ts; incidentSync.ts; apps/web/public/sw.js
Deployment and mobile	docker-compose.yml; render.yaml; both Dockerfiles; services/api/start.sh; apps/web/next.config.mjs; Capacitor/Android files
Quality and demo data	.github/workflows/ci.yml; services/api/tests/; web security tests; services/api/app/seed.py; login page
Legal and safety	LICENSE; DISCLAIMER.md

Repository reference: [ResQGrid AI source](#). Framework/provider reference sites: [Next.js](#), [FastAPI](#), [MapLibre GL JS](#), [Capacitor](#), [Gemini API](#), and [Alibaba Cloud Model Studio](#). Consult current terms and documentation before configuring external services; this guide does not reproduce or replace them.

The project LICENSE states Copyright (c) 2026 ResQGrid AI Team, All Rights Reserved. The software and associated documentation are proprietary; unauthorized copying, distribution, modification, or use is prohibited, and prior written permission is required. The full LICENSE and DISCLAIMER remain controlling documents. This guide does not grant additional rights or imply an official partnership/certification.

To maintain the package, update `docs/PROJECT_DOCUMENTATION.md` and `docs/DOCUMENTARY_SCRIPT.md`, revise the source baseline when warranted, and run `python docs/generate_documents.py` from the repository root. The generator requires `python-docx` and `ReportLab`; its validation step additionally uses `PyMuPDF`. It exports both sources to PDF and DOCX in `docs/exports`. Run `python docs/generate_documents.py --check` to validate existing exports without regenerating them. Install documentation-only dependencies in an isolated environment if needed; they are not new application runtime dependencies.

Exports include navigable chapter contents, formatted tables/code, and page numbers. Word pagination may vary by installed fonts and Word version. Regeneration overwrites only the four named export files. Keep screenshots and examples synthetic, refresh statements when behavior changes, and do not treat a new document export as authorization to commit, publish, or redeploy the application.